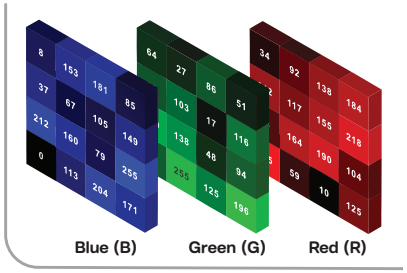
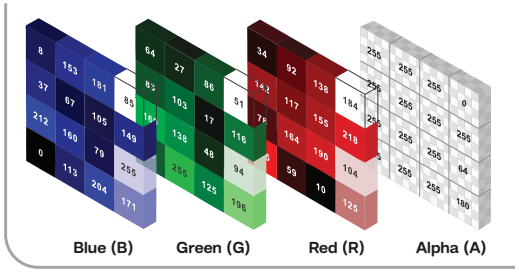
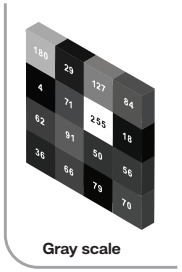
DATA STRUCTURE WITH OPENCV

8-bit unsigned integer (0-255)







Made by:

Gabriel Martínez-Soltero
Lucila Jiménez R.

INSTALLATION AND USE
pip install opencv-python
import cv2

I/O	
img=cv2.imread("path of the image",flag)	Read the image on the path, flag can be: cv2.IMREAD_GRAYSCALE cv2.IMREAD_UNCHANGED
cv2.imshow("Window's name", img)	Creates a window to show the image
cv2.imwrite("name.png",img)	Saves the image img
cap=cv2.VideoCapture(0) cap=cv2.VideoCapture("rtsp://a:p@192.168.1.1:554/ft") cap=cv2.VideoCapture("path of the video")	Opens the camera or the video
ret, frame=cap.read()	Reads a frame from camera
cap.release()	Releases camera
fourcc=cv2.VideoWriter_fourcc(*'mp4v') out=cv2.VideoWriter("res.mp4", fourcc, fps, (w, h))	Definition of the codec and video
out.write(frame)	Writes the frame on the file
out.release()	Releases the writer and file is created
destroyAllWindows()	Releases and closes all windows
cv2.waitKey(1000)	Waits a second for a keypress

PROPERTIES	
img.size	Returns number of elements of img
img.shape	Returns dimensions of the img
img.dtype	Returns type of elements
img.nbytes	Returns number of bytes of img

GEOMETRIC	
img=cv.2.resize(img,(width,height),)	Resizes img at specified dimensions
img=cv2.flip(img,axis)	Flips the image over the axis. 0 for X, 1 for Y and -1 for both
img=cv2.rotate(img, cv2.ROTATE_180)	Rotates img acording to cv2.ROTATE_180 cv2.ROTATE_90_CLOCKWISE cv2.ROTATE_90_COUNTERCLOCKWISE
roi=img[initial_row:limit_row,initial_col:limit_col]	Returns a portion of the image

COLOR SPACES	
img=cv2.cvtColor(img, cv2.COLOR_BGR2HSV)	Change the color space acording to the change cv2.COLOR_BGR2GRAY cv2.COLOR_BGR2BGRA cv2.COLOR_BGR2RGB cv2.COLOR_BGR2HSV cv2.COLOR_BGR2YCrCb cv2.COLOR_BGR2LAB cv2.COLOR_HSV2BGR
b,g,r=cv2.split(img)	Split the image in the available channels
img=cv2.merged([b,g,r])	Merges the array of channel into a image

ARITHMETIC OPERATIONS	
img=cv2.add(img1,img2)	Adds img1 + img2 handling overflow
img=cv2.subtract(img1,img2)	Subtracts img1 -img2 handling overflow
img=cv2.absdiff(img1, img2)	Applies absolute value to the difference
img=cv2.addWeighted(img1, alpha, img2,beta, gamma)	Applies a weighted sum where img1*alpha+img2*beta+ gamma weights are float values between 0 and 1

LOGIC OPERATIONS	
img=cv2.bitwise_not(img)	Performs not operator at bit level
img=cv2.bitwise_and(img)	Performs and operator at bit level
img=cv2.bitwise_or(img)	Performs or operator at bit level
img=cv2.bitwise_xor(img)	Performs xor operator at bit level

SEGMENTATION	
_,th=cv2.threshold(img_gray,thresh,max_val,type)	Compares each pixel against a threshold and assigns it a new value according to a logical rule. cv2.THRESH_BINARY cv2.THRESH_BINARY_INV cv2.THRESH_TRUNC cv2.THRESH_TOZERO cv2.THRESH_TOZERO_INV
mask=cv2.inRange(img,minc,maxc)	Creates a binary image. If the pixel is within the specified range, it becomes 255; otherwise 0. minc and maxc are numpy arrays with a element for each channel

DRAWING	
img=cv2.line(img,pt0,pt1,color,thickness)	Draws a line from pt0 (x0,y0) to pt1 (x1,y1) of the color specified by the (b,g,r) values and with the thickness defined in pixels
img=cv2.rectangle(img,pt0,pt1,color,thickness)	Draws a rectangle
img=cv2.circle(img,center,radius,color,thickness)	Draws a circle (radius is defined in pixels)
img=cv2.polylines(img,pts,closed,color,thickness)	Draws multyple lines connecting all pts which are defined as a numpy array of shape (n,1,2) where n is the number of points, if closed is True a line that connects the first point with the last will be draw
img=cv2.putText(img,"text",position, font,font_scale,color,thickness)	Places the text at certain position, some of the fonts are: cv2.FONT_HERSHEY_SIMPLEX cv2.FONT_HERSHEY_DUPLEX cv2.FONT_HERSHEY_TRIPLEX cv2.FONT_HERSHEY_COMPLEX cv2.FONT_HERSHEY_PLAIN cv2.FONT_HERSHEY_COMPLEX_SMALL cv2.FONT_HERSHEY_SCRIPT_COMPLEX cv2.FONT_HERSHEY_SCRIPT_SIMPLEX cv2.FONT_ITALIC